CH-6 Part-5

In-Depth Study Notes: Advanced Self Joins, Peer-to-Peer Comparisons, and Deduplication

As an expert Oracle SQL Developer, mastering **Self Joins** allows you to unlock complex peer-to-peer data analysis without relying on resource-heavy subqueries. The provided execution log focuses on advanced techniques for querying ranges based on dynamic anchor rows, finding unique colleague mat rices, using function-based join conditions, and blending self-joins with non-equijoins.
Below is a detailed technical breakdown of these advanced SQL architectures.

Concept 1: Peer-to-Peer Range Comparisons (Dual Anchor Joins)

Core Purpose: This architecture dynamically filters a dataset to find rows that fall between the values of two specific "anchor" records within the *same* table 1, 2. Instead of hardcoding bounds (e.g., BETWEEN '01-JAN-95' AND '31-DEC-99'), the bounds are derived dynamically based on employee names or IDs.
Keyword Breakdown:

- **JOIN ... ON**: Used twice to conceptually duplicate the table into two separate "Anchor" instances (e.g., Anchor 1 is Mourgos, Anchor 2 is Higgins) 2.
- **BETWEEN ... AND**: Evaluates if the target row's value falls inside the range of the two anchors 1, 2.
- **OR**: **Critical logical operator here.** The BETWEEN operator *strictly* requires the lower limit to be declared first. Because we do not know programmatically if Anchor 1's value is higher or lower than Anchor 2's value, we must provide an OR condition reversing the bounds to prevent empty result sets 1, 2.

Syntax Breakdown:

```
SELECT target_alias.column_list
FROM table_name target_alias
JOIN table_name anchor1 ON anchor1.identifier_col = 'Value1'
JOIN table_name anchor2 ON anchor2.identifier_col = 'Value2'
WHERE target_alias.numeric_or_date_col BETWEEN anchor1.col AND
anchor2.col
   OR target_alias.numeric_or_date_col BETWEEN anchor2.col AND
anchor1.col;
```

Detailed Example: To find all employees hired in the time gap between Mourgos and Higgins:

```
SELECT O.LAST_NAME OTHER, O.HIRE_DATE
FROM EMPLOYEES O
JOIN EMPLOYEES MO ON MO.LAST_NAME = 'Mourgos'
JOIN EMPLOYEES HI ON HI.LAST_NAME = 'Higgins'
WHERE O.HIRE_DATE BETWEEN MO.HIRE_DATE AND HI.HIRE_DATE
   OR O.HIRE_DATE BETWEEN HI.HIRE_DATE AND MO.HIRE_DATE;
```

Execution outputs 13 employees whose hire dates successfully fall into that calculated temporal gap 2, 3.

Execution Logic:

1. **Virtualization:** The engine creates three instances of EMPLOYEES in memory (O, MO, HI) 2.
2. **Anchor Filtering:** It filters MO down to Mourgos's row and HI down to Higgins's row during the join phase 2.
3. **Range Evaluation:** It scans the O (Other) instance row-by-row, evaluating the OR BETWEEN boolean logic 2.

Pro-Tips/Best Practices:

- **Performance Impact:** Ensure the anchor identifier columns (e.g., LAST_NAME) are indexed. If they are not, the engine must perform three separate Full Table Scans to build the virtual instances.

Concept 2: Deduplicating Self Join Pairs (The Unique Colleague Matrix)

Core Purpose: When you join a table to itself to find "peers" (e.g., two employees in the exact same department), a standard Self Join produces redundant data 3. It will match Employee A to Employee A (self-match), Employee A to Employee B, and Employee B to Employee A (mirror match) 3-6. To generate a clean matrix of unique colleagues, you must use primary key inequalities 7, 8.

Keyword Breakdown:

- **< or > Operators**: Placed in the ON or WHERE clause comparing Primary Keys (e.g., E.EMPLOYEE_ID < C.EMPLOYEE_ID) 8. This mathematically forces the engine to only register a pair *once* in a single direction, instantly eliminating both self-matches and mirror matches 8, 9.
- **<> or != Operators**: Placed in the ON clause to eliminate self-matches, but this will *still* allow mirror matches (A-B and B-A) 7, 10.

Comparison of Self Join Deduplication Logic:

Join Condition Constraint,"Resulting Pairs (Example: Lex(102), Alexander(103))",Business Use Case
E.DEPT_ID = C.DEPT_ID (No PK filter),"Lex-Lex, Lex-Alexander, Alexander-Lex, Alexander-Alexander 5",Raw Cartesian matches (rarely useful) 5.
E.EMP_ID <> C.EMP_ID,"Lex-Alexander, Alexander-Lex","Bidirectional matching (e.g., finding all colleagues of a person) 7, 10."
E.EMP_ID < C.EMP_ID,Lex-Alexander 8,"Unique Pair Generation (e.g., creating 1-to-1 mentorship pairings) 8, 9."

Syntax Breakdown:

```
SELECT alias1.col, alias2.col
FROM table_name alias1
JOIN table_name alias2
  ON alias1.shared_attribute = alias2.shared_attribute
 AND alias1.primary_key < alias2.primary_key;
```

Detailed Example: Generate a clean, deduplicated list of colleagues sharing the exact same DEPARTMENT_ID:

```
SELECT E.FIRST_NAME EMP_NAME, E.DEPARTMENT_ID, C.DEPARTMENT_ID,
C.FIRST_NAME DEPT_COLL_NAME
FROM EMPLOYEES E
JOIN EMPLOYEES C
  ON E.DEPARTMENT_ID = C.DEPARTMENT_ID
 AND E.EMPLOYEE_ID < C.EMPLOYEE_ID;
```

Execution outputs 21 unique rows, dropping from the initial 61 raw matches 8, 11.

Execution Logic:

1. The engine creates virtual tables E and C 11.
2. It matches rows where DEPARTMENT_ID is equal 11.
3. Before projecting, it evaluates E.EMPLOYEE_ID < C.EMPLOYEE_ID. If Steven (ID 100) matches Neena (ID 101), 100 < 101 evaluates to TRUE, and the row is kept 8. If Neena (101) matches Steven (100), 101 < 100 evaluates to FALSE, and the duplicate mirror row is discarded 8.

Concept 3: Function-Based Peer Joins

Core Purpose: Sometimes relationships do not exist in raw columns but in derived data. A Function-Based Self Join applies a single-row function (like converting a date to a quarter) on both virtual tables to find matching peers based on that calculated value 12.

Keyword Breakdown:

- **TO_CHAR(date, 'fmt')**: Used specifically here to convert the HIRE_DATE into a single numeric Quarter digit ('Q'), allowing the join to evaluate temporal groupings (e.g., Q1, Q2, Q3, Q4) 12.

Syntax Breakdown:

```
SELECT alias1.col, alias2.col
FROM table_name alias1
JOIN table_name alias2
  ON function_name(alias1.col) = function_name(alias2.col)
 AND alias1.pk < alias2.pk;
```

Detailed Example: To find employees who were hired in the *same quarter* of the year (regardless of the specific year), deduplicated:

```
SELECT E.FIRST_NAME EMP_FIRST_NAME, C.FIRST_NAME
HIRE_QTR_COLL_FIRST_NAME
FROM EMPLOYEES E
JOIN EMPLOYEES C
  ON TO_CHAR(C.HIRE_DATE, 'Q') = TO_CHAR(E.HIRE_DATE, 'Q')
 AND E.EMPLOYEE_ID < C.EMPLOYEE_ID;
```

Execution outputs 49 unique pairs of colleagues who share the same hiring quarter 12, 13.

Execution Logic:

1. The engine passes the HIRE_DATE of every row in virtual table E through the TO_CHAR function to calculate the quarter in memory 12.
2. It does the same for virtual table C 12.
3. It joins the rows where the calculated Quarter values are strictly equal, then applies the Primary Key deduplication filter 12.

Pro-Tips/Best Practices:

- **Function-Based Indexes:** Using functions like TO_CHAR directly in an ON clause will completely bypass standard B-Tree indexes, forcing a Full Table Scan. If this is a frequent production query, you must create a **Function-Based Index** on TO_CHAR(HIRE_DATE, 'Q') to maintain database performance.

Concept 4: Multi-Table Non-Equi Self Joins

Core Purpose: This is a highly complex architecture combining a Self Join with an intermediate Range/Lookup table 14, 15. It is used when the "shared attribute" between two peers isn't a direct value, but rather a categorized tier (like a Salary Grade or Tax Bracket) determined by evaluating a third table 15.

Keyword Breakdown:

- **JOIN ... ON ... BETWEEN ... AND**: This non-equijoin syntax must be executed *twice*. Once to map Employee A to a Grade, and once to map Employee B to that same Grade 15.

Syntax Breakdown:

```
SELECT e_alias.col, c_alias.col
FROM main_table e_alias
JOIN lookup_table ON e_alias.numeric_col BETWEEN lookup_table.min_val
AND lookup_table.max_val
JOIN main_table c_alias ON c_alias.numeric_col BETWEEN
lookup_table.min_val AND lookup_table.max_val
WHERE e_alias.pk < c_alias.pk;
```

Detailed Example: To find unique pairs of colleagues who fall into the exact same Salary GRADE_LEVEL (from the JOB_GRADES table), even if their actual numeric salaries are different:

```
SELECT E.LAST_NAME EMP_LAST_NAME, C.LAST_NAME
GRADE_LEVEL_COLL_FIRST_NAME
FROM EMPLOYEES E
JOIN JOB_GRADES ON E.SALARY BETWEEN LOWEST_SAL AND HIGHEST_SAL
JOIN EMPLOYEES C ON C.SALARY BETWEEN LOWEST_SAL AND HIGHEST_SAL
WHERE E.EMPLOYEE_ID < C.EMPLOYEE_ID;
```

Execution correctly drops from 90 non-deduplicated matches down to 35 unique colleague pairings 15, 16.

Execution Logic:

1. **Phase 1 (Target Mapping):** The engine takes Employee E and evaluates their salary against the JOB_GRADES tiers, attaching the correct tier to the E row 15.
2. **Phase 2 (Peer Mapping):** The engine takes Employee C and maps their salary against the same JOB_GRADES table instance 15.
3. **Phase 3 (Intersection):** Because both E and C are joined to the *same instance* of the JOB_GRADES row, they are intrinsically linked together as peers sharing that grade 15.
4. **Phase 4 (Filter):** The engine executes the < Primary Key logic to output the final 35 unique pairs 15, 16.

Pro-Tips/Best Practices:

- **Syntax Strictness:** As demonstrated in the source log, misspelled keywords like BETWEN or BEWEEN will instantly halt compilation and throw an **ORA-00920: invalid relational operator** error 13. Always double-check your logical operators in deep multi-table joins.